Daily Quant Digest: 2026-11-06

Foundations Restart, day 11. Three new steps today: the second half of the classic puzzles, **the birthday problem** (why 23 people are enough) and **boy-or-girl** (why “at least one” and “this one” are different news); **repo**, the giant pawn shop where the world’s bonds are turned into cash every night; and **binary search on the answer**, where we stop searching a list and start searching for the best number itself.

Today’s topics

Track	Topic	Difficulty
Probability & Statistics	FR Lesson 10 (part 2 of 2): the birthday problem and boy-or-girl	Easy-Medium
Derivatives	FR Lesson 11: repo, borrowing against bonds	Easy-Medium
Algorithms & DS	FR Lesson 11: binary search on the answer: the slowest safe execution rate	Medium
Music	Creedence Clearwater Revival, “Fortunate Son” (1969)	

1. Probability & Statistics: The Birthday Problem and Boy-or-Girl

> **Where we are.** Lessons 3 and 4: counting arrangements and choices, including $\binom{n}{2}$ for “how many pairs”. Lesson 2 and Lesson 5: the **complement trick**, “count what you do not want, then subtract from 1”. Lesson 6: news **moves the walls in**. Lesson 10 part 1 (yesterday): Monty Hall, where the *rule* that produced the news mattered. Today: one puzzle that is pure counting plus the complement, and one that is yesterday’s lesson again in a new costume.

1.1 The birthday problem: the surprise

How many people must be in a room before there is a better than even chance that two of them share a birthday? Ignore 29 February and assume all 365 days are equally likely.

Most people guess around 180 (half of 365). The answer is **23**. With 23 people the chance is 50.7%. With 50 people it is 97%.

1.2 The central picture: handshakes, not people

The mistake is to think about **people**. The thing that can match is a **pair** of people. So picture everyone in the room shaking hands with everyone else. Each handshake is one small lottery ticket: "do these two share a birthday?" Each ticket wins with chance only $1/365$. But the number of tickets grows much faster than the number of people.

In the picture	In the maths
A person in the room	One of n people, each with a random birthday
A handshake	A pair, $\binom{n}{2}$ of them (Lesson 4)
A handshake "wins"	That pair shares a birthday, chance $1/365$
Nobody's handshake wins	All n birthdays are different

With 23 people there are $\binom{23}{2} = \frac{23 \times 22}{2} = 253$ handshakes. That is 253 lottery tickets, not 23. Now a 50% chance feels much less magic.

Where the picture stops working: the tickets are not fully independent (Lesson 7). If Amy matches Ben and Ben matches Cat, then Amy matches Cat for sure. So we cannot just multiply ticket by ticket. For the exact answer we count instead.

1.3 Count it exactly: people walk in one at a time

"At least one match" is messy: one pair could match, or two pairs, or three people on one day. Use the complement (Lesson 2): find the chance that **all birthdays are different**, then subtract from 1.

Let people walk in one at a time and ask, each time, "is your birthday new?"

- Person 1: any day is fine. Chance $365/365$.
- Person 2: must avoid 1 used day. Chance $364/365$.
- Person 3: must avoid 2 used days. Chance $363/365$.
- Person n: must avoid $n-1$ used days. Chance $(365-n+1)/365$.

Multiply along the path (Lesson 6, the multiplication rule):

$$P(\text{all different}) = \frac{365}{365} \times \frac{364}{365} \times \frac{363}{365} \times \cdots \times \frac{365-n+1}{365}.$$

Read it out loud: "the first person is free, and each new person has one fewer free day than the person before." This is also pure counting: the top is the number of ways to give n people **different** days in order, $365 \times 364 \times \cdots$ (Lesson 3, arrangements), and the bottom 365^n is the number of ways to give them **any** days.

Then $P(\text{at least one match}) = 1 - P(\text{all different})$.

Small numbers first. With 4 people: $1 \times 0.99726 \times 0.99452 \times 0.99178 = 0.98364$. So the chance of a match is only 1.6%. Each new person costs a little, but the costs grow, and they pile up.

People n	**Handshakes** $\binom{n}{2}$	$P(\text{match})$
5	10	2.7%
10	45	11.7%
20	190	41.1%
22	231	47.6%
23	**253**	**50.7%**
30	435	70.6%
50	1,225	97.0%
70	2,415	99.9%

Imagine 1,000 offices, each with exactly 23 people. In about **507** of them, two colleagues share a birthday.

A quick estimate using the handshakes. Pretend the 253 tickets were independent. Then the chance none wins is $(364/365)^{253} = 0.4995$, so a match has chance about 0.5005. That is very close to the exact 0.5073. The handshake picture is not exact, but it gets you the right answer in your head.

1.4 The twist: someone sharing *your* birthday

A different question: how many **other** people do you need before one of them shares **your** birthday? Now only the handshakes that include **you** count. With m others there are only m such handshakes, and each is a separate person, so they really are independent:

$$P(\text{nobody has my birthday}) = \left(\frac{364}{365}\right)^{m}.$$

This drops below $1/2$ only at $m = 253$. You need **253 other people**. Same number as the handshakes in a room of 23, and that is no accident: in both cases you need about 253 tickets.

1.5 Boy-or-girl: “at least one” is not “this one”

A family has two children. Each child is a boy or a girl, half and half, independently (Lesson 7). Imagine **1,000 such families**:

Older, younger	Families
Boy, Boy	250
Boy, Girl	250
Girl, Boy	250
Girl, Girl	250

Now three different pieces of news. Each one moves the walls in a different way (Lesson 6).

News 1: "At least one is a boy." The room keeps BB, BG and GB: 750 families. Both boys in 250. So $P(\text{two boys}) = 250/750 = 1/3$.

News 2: "The older one is a boy." The room keeps BB and BG: 500 families. So $P(\text{two boys}) = 250/500 = 1/2$.

News 3: you meet the father in the park with one child, a boy. He picked which child to bring by a coin flip. In BB families he brings a boy every time: 250. In BG and GB families he brings the boy half the time: 250 out of 500. In GG families, never. The room holds $250 + 250 = 500$ families and 250 are BB: $1/2$ again.

Why is News 1 different? "At least one boy" throws out only the GG families and keeps all three mixed-or-boy families **at full weight**. News 2 and 3 point at **one particular child**. Once a particular child is a boy, the other child is just a fresh coin: 1/2.

This is yesterday's lesson in a new costume: **the words "I saw a boy" are not enough; you need the rule that produced them.** In Monty Hall the host's forced choice gave likelihood ratio 2. Here, in News 3, a BB family is **twice as likely** to show you a boy as a mixed family (1 vs 1/2), but there are twice as many mixed families. Prior odds BB : mixed $= 1 : 2$, times likelihood ratio 2, gives $1 : 1$. Probability $1/2$ (Lesson 9, odds form).

1.6 Common mistakes

- **Birthday: thinking about people, not pairs.** The room of 23 has 253 chances to match.
- **Birthday: confusing "any two match" with "someone matches me".** 23 people vs 253 people.
- **Boy-or-girl: treating "BG" and "GB" as one case.** Then you get BB, mixed, GG, each 1/3, which is wrong. Mixed families are twice as common. Always label the children (older, younger) so the outcomes are equally likely (Lesson 1).
- **Boy-or-girl: assuming the answer from the words alone.** "One of them is a boy" can mean News 1 or News 3. In an interview, say which rule you are assuming.

1.7 Self-check

1. In a room of 3 people, what is the exact chance that at least two share a birthday?

2. A family has two children and you learn “at least one is a girl”. What is the chance both are girls?
3. Why is the answer to “someone shares *my* birthday” (253) so much bigger than 23?

1.8 Interview question: birthdays on the trading floor

(An Optiver / SIG style warm-up. The interviewer wants the complement trick, quick estimates, and care about how information was produced.)

a. A desk has 10 traders. What is the chance at least two share a birthday? First estimate it in your head, then give the exact value.

b. Ignore days and only look at **birth months** (12 equally likely months). What is the smallest team where the chance that two share a month is over 50%?

c. You join a firm. How many colleagues does the firm need before the chance that at least one of them shares *your* birthday is over 50%?

d. A colleague has **three** children and says, in answer to “Do you have a daughter?”, “Yes.” What is the chance all three are girls?

e. Another colleague with three children brings one, chosen at random, to the office party. It is a girl. What is the chance all three are girls?

1.9 Solution

(a) Estimate: $\binom{10}{2} = 45$ handshakes, each with chance $1/365$, so about $45/365 \approx 12\%$. Exact, with the walk-in method:

$$P(\text{all different}) = \frac{365 \times 364 \times \cdots \times 356}{365^{10}} = 0.8831, \qquad P(\text{match}) = 11.7\%.$$

The estimate is a little high because it counts some matches twice, but it is close.

(b) Same method with 12 instead of 365.

Team size	$P(\text{all months different})$	$P(\text{match})$
3	$\frac{12\cdot11\cdot10}{12^3} = \frac{1320}{1728}$	23.6%
4	$\frac{12\cdot11\cdot10\cdot9}{12^4} = \frac{11880}{20736}$	42.7%
5	$\frac{12\cdot11\cdot10\cdot9\cdot8}{12^5} = \frac{95040}{248832}$	**61.8%**

Answer: **5 people**. (With 5 people there are 10 handshakes, each with chance $1/12$: the quick estimate says “close to a coin flip or better”, which agrees.)

(c) Each colleague misses your birthday with chance $364/365$, independently. We need $(364/365)^m < 1/2$.

- $m = 252$: $(364/365)^{252} = 0.5009$, still above a half.

- $m = 253$: $(364/365)^{253} = 0.4995$, below a half.

Answer: **253 colleagues**. Quick estimate: each colleague "costs" you about $1/365$ of the "no match" chance, and to get down to one half you need about $0.693 \times 365 \approx 253$ of them. (The 0.693 is the same number behind the rule of 72 in Derivatives Lesson 1: halving and doubling use the same number.)

(d) Label the children oldest to youngest. There are $2^3 = 8$ equally likely families (Lesson 3). "Yes, I have a daughter" (at least one girl) removes only BBB, leaving 7. Only GGG is all girls:

$$P(\text{GGG} \mid \text{at least one girl}) = \frac{1}{7}.$$

(e) Now the news points at a **particular** child. Use 800 families, 100 of each type. A family with g girls brings a girl with chance $g/3$.

Girls in family	Families	Bring a girl
0	100	0
1	300	100
2	300	200
3	100	100
Total	800	**400**

Of the 400 families who brought a girl, 100 are GGG: $P = 100/400 = 1/4$. That is the same as "the other two children are fresh coins": $\frac{1}{2} \times \frac{1}{2} = \frac{1}{4}$.

Interview trick. For birthday-type questions: count the pairs, multiply by the chance per pair, and you have a quick estimate; then do the exact walk-in product if asked. For boy-or-girl questions, ask: "does the news point at one particular child?" If yes, the others are fresh coins. If it is only "at least one", count the families.

Sanity checks. (a) The exact answer is just below the pair estimate, as it should be. (b) 4 people give under 50% and 5 give over. (d) $1/7$ is smaller than $1/4$ in (e), because "at least one girl" keeps all the one-girl families at full weight. All numbers were checked by exact computation.

Self-check answers. (1) $1 - \frac{364}{365} \cdot \frac{363}{365} = 1 - 0.99180 = 0.82\%$. (2) $1/3$: the room is GG, GB, BG. (3) In a room of 23 every pair counts (253 pairs); for *your* birthday only the pairs that include you count, one per colleague.

---

2. Derivatives: Repo, or the Pawn Shop for Bonds

> **Where we are.** Lesson 1: interest on a deposit, with ACT/360 money market rates. Lesson 2: present value. Lesson 3: long, short and leverage. Lesson 8: bond prices and the price-yield seesaw. Lesson 9: DV01, how much a bond moves for 1 bp. Lesson 10 (yesterday): Treasuries and how they are quoted. Today: how the people who hold those bonds pay for them.

2.1 The scene

It is 5pm in New York. A bond dealer (say the Treasury desk of a big bank) has just bought **USD 100m** of a new 10-year Treasury note from a client. It must pay for it tonight. It does not have USD 100m of spare cash. Its business is buying and selling bonds, not sitting on piles of money.

Across town, a **money market fund** holds billions of dollars of savers' cash. It wants a safe place to park that cash overnight and earn a little interest. It does not want to buy a 10-year bond, because those swing too much (Lesson 9).

The answer for both is a **repo**. The dealer gives the fund the bond tonight and takes cash. Tomorrow the dealer pays back the cash plus a little interest and takes the bond back. The dealer has financed its bond; the fund has lent its cash with a bond as safety. Trillions of dollars are done like this every single day. It is the plumbing under the whole bond market.

2.2 The central picture: a pawn shop

You own a gold watch. You need cash for a week. You walk into a pawn shop, hand over the watch and get cash. A week later you return, pay back the cash plus a fee, and take your watch home. If you never come back, the shop sells the watch.

In the picture	In the maths
You, needing cash	The dealer (the **repo** side: it "sells and repurchases")
The pawn shop with cash	The money market fund (the **reverse repo** side)
The gold watch	The bond, called the **collateral**
The shop lends less than the watch is worth	The **haircut**
The fee for the week	Interest at the **repo rate**
The shop sells the watch if you vanish	The lender sells the bond if the borrower defaults
Coming back next day / next week	**Overnight** repo / **term** repo

The name “repo” is short for **sale and repurchase agreement**. Legally, the dealer *sells* the bond today and agrees to *buy it back* tomorrow at a fixed higher price. Economically, it is a loan with the bond as security. The difference between the two prices is the interest.

Where the analogy stops: a pawn shop lends at a high fee to people it does not trust. Repo is the opposite: because the collateral is a US Treasury, about the safest thing there is, the repo rate is usually one of the **lowest** borrowing rates in the market, just around the central bank’s policy rate.

2.3 The mechanics, one at a time

Two legs. The **start leg**: bond goes one way, cash the other. The **end leg**: the cash comes back with interest, the bond comes back. Both prices are fixed on day one.

The repo rate. Quoted like a money market deposit (Lesson 1), ACT/360:

$$\text{interest} = \text{cash lent} \times \text{repo rate} \times \frac{\text{days}}{360}.$$

Read it out loud: “the cash, times the yearly rate, times the fraction of a 360-day year”. The repurchase price is the cash plus this interest.

The haircut. The lender gives a bit less cash than the bond is worth, say 98% for a 2% haircut. Why? If the borrower disappears overnight and the bond has fallen in price, the lender must still be able to sell it and get all its cash back. The haircut is the lender’s cushion. Safer bonds get smaller haircuts.

Who keeps the bond’s ups and downs? The **borrower** does. If the bond falls, the dealer still has to buy it back at the agreed price. So the dealer keeps all the price risk, and also gets any coupon paid during the repo (the lender passes it back). The lender only cares that the cash comes back.

General collateral vs special. Most days, the fund does not care *which* Treasury it holds: any will do. That is **general collateral (GC)**, with one market rate. But sometimes one particular bond is in great demand, usually because many traders have sold it short and need to borrow it to deliver it (Lesson 3: to go short you must borrow what you sell). Then people who own that bond can borrow cash against it at a **lower** rate, because the lender is really paying to get that bond. That bond is **“special”**. A lower repo rate is a reward for owning the bond everyone wants.

Link to SOFR and to FX. The main US short-term interest rate, **SOFR** (Secured Overnight Financing Rate), is simply the average rate on overnight Treasury repo. And you already know a close cousin: an **FX swap** (Lesson 6) is a repo where the collateral is another currency. You hand over EUR today and get USD, and you swap back later at an agreed rate. The forward points play the role of the repo interest.

2.4 Worked example: financing a 10-year note for a month

The dealer buys USD 100m face of a newly issued 10-year note with a 4.25% coupon at a price of **99-16** (Lesson 10: $99 + 16/32 = 99.50$). It was just issued, so we ignore accrued interest. The rates below are illustrative.

Step 1: what must be paid. $100{,}000{,}000 \times 0.995 = \text{USD } 99{,}500{,}000$.

Step 2: the repo. 30-day term repo, rate 3.90%, haircut 2%.

- Cash borrowed: $0.98 \times 99{,}500{,}000 = \text{USD } 97{,}510{,}000$.
- The dealer's own money (equity): $99{,}500{,}000 - 97{,}510{,}000 = \text{USD } 1{,}990{,}000$.
- Leverage: $99.50 / 1.99 = 50$ times (Lesson 3: like a mortgage with a 2% deposit).

Step 3: the interest.

$$97{,}510{,}000 \times 0.039 \times \frac{30}{360} = \text{USD } 316{,}907.50.$$

The repurchase price in 30 days is $97{,}510{,}000 + 316{,}907.50 = \text{USD } 97{,}826{,}907.50$. Overnight instead of 30 days, it would be $97{,}510{,}000 \times 0.039/360 = \text{USD } 10{,}563.58$ a night.

Step 4: the carry. While it holds the bond, the dealer earns the coupon. 4.25% a year on USD 100m is about $100{,}000{,}000 \times 0.0425 \times 30/365 \approx \text{USD } 349{,}300$ for 30 days. So:

Item	USD
Coupon earned (about)	+349,300
Repo interest paid	−316,900
Carry	**about +32,400**

This is called **positive carry**: the bond pays more than its financing costs. On equity of USD 1.99m, that is about 1.6% in a month. Leverage makes small gaps look big.

Step 5: the risk. Now the seesaw (Lesson 8). This note's DV01 is about **USD 80,300 per bp** on USD 100m (Lesson 9). If the 10-year yield rises 10 bp:

$$10 \times 80{,}300 \approx \text{USD } 803{,}000 \text{ loss } (\text{exact repricing gives } 799{,}158).$$

That is about **40% of the dealer's equity**, wiped out by a normal week's move, against a month's carry of USD 32,400. A rise of about $1{,}990{,}000/80{,}300 \approx 25$ bp would eat the whole haircut. That is exactly the cushion the fund wanted: 25 bp is a big move for one night.

Step 6: the lender's view. The fund lent USD 97.51m and gets back USD 97,826,907.50, holding a Treasury worth USD 99.5m the whole time. If the dealer fails, the fund sells the note. As long as it has not fallen more than about 2%, the fund is whole.

Bonus: a special. If this note went special at 3.20%, the 30-day interest would drop to $97{,}510{,}000 \times 0.032 \times 30/360 = \text{USD } 260{,}026.67$, and the carry would jump from about 32,400 to about 89,300. Owning the bond everyone wants to borrow pays.

2.5 When the plumbing broke: September 2019

Repo usually trades within a few basis points of the Fed's target. On **17 September 2019** it did not. A big corporate tax payment date and a large Treasury settlement drained cash from the system on the same day, and banks were not willing to lend out their reserves. Overnight repo rates spiked, briefly trading as high as about 10% intraday, and SOFR printed 5.25%, far above the Fed's target range. The Fed stepped in with its own repo operations, lending cash against Treasuries, for the first time in about a decade. The lesson: repo is cheap and boring until the cash is suddenly not there, and a dealer financing at 50 times leverage overnight must roll that loan **every single day**. In 2008, firms like Bear Stearns found that when lenders stop rolling repo, a bank can fail within days.

2.6 Likely interview questions

1. **"Why is the repo rate usually below unsecured interbank rates?"** Because the loan is backed by Treasuries plus a haircut. If the borrower fails, the lender sells the bond. Less risk, lower rate.
2. **"What does it mean when a bond trades special, and who benefits?"** Its repo rate is below GC, usually because many people are short it and need to borrow it. Owners of the bond can borrow cash cheaply against it; short sellers earn less on the cash they lend.
3. **"You repo USD 50m of cash at 4.00% for 7 days. What do you get back?"** $50{,}000{,}000 \times (1 + 0.04 \times 7/360) = \text{USD } 50{,}038{,}888.89$.

Self-check. (1) In a repo, who keeps the bond's price risk? (2) Why would a lender ask for a bigger haircut on a 30-year bond than on a 3-month bill? (3) If the coupon is 4.25% and term repo is 4.50%, is carry positive or negative?

Answers. (1) The borrower (the dealer), because it must buy the bond back at the fixed price. (2) A 30-year bond can fall much more in one night (higher DV01), so the lender needs a bigger cushion. (3) Negative: financing costs more than the coupon pays (ignoring the small gap from the bond's price being below 100).

---

3. Algorithms & Data Structures: Binary Search on the Answer

> **Where we are.** Lesson 1: Big-O. Lesson 5: running totals. Lesson 10 (yesterday): binary search as a **red/green fence**: if a yes/no question is "no, no, …, yes, yes" along a line, we find the first "yes" in about $\log_2 n$ looks, with `firstTrue(lo, hi, pred)`. Today

> the line of fence posts is not a list in memory. It is **the possible answers themselves**.

3.1 The central picture: a dial on a machine

Picture a machine with a speed dial from 1 to a million. Turn it up and the job finishes sooner; turn it down and it takes longer. You want the **lowest** setting that still finishes in time. You cannot compute that setting directly. But for any setting, you can run a quick test: "at this speed, do I finish in time?"

That test is a fence along the dial:

In the picture	In the code
The dial settings $1 \dots 10^9$	The range of possible answers $[lo, hi]$
"At this speed, do I finish in time?"	`hoursNeeded(q, k) <= H`
Too slow, too slow, …, fast enough, fast enough	false, …, false, true, …, true
The lowest setting that works	The first true post

Why is it a fence? Because faster never hurts: if speed 7 finishes in time, so does speed 8. That one property, called **monotone** ("only goes one way"), is all binary search needs.

So the recipe for "binary search on the answer" has three steps:

1. Find the range of possible answers, $[lo, hi]$.
2. Write a **checker**: given a candidate answer, is it good enough?
3. Prove the checker is a fence (once good, always good), then run yesterday's `firstTrue`.

Interviewers love this pattern because the words "minimise the maximum", "smallest capacity", "least speed" or "fewest days" are a strong hint, and the checker is often a simple greedy loop.

3.2 The problem: the slowest safe execution rate

You must work n block orders with sizes $q_0, \dots, q_{n-1}$ (in lots) within H hours. Each hour, the execution algo works **one** order and fills up to k lots of it. If that order has fewer than k lots left, it finishes the order and sits idle for the rest of the hour (it never splits an hour between two orders). Going faster moves the market against you, so you want the **smallest integer** k that finishes everything within H hours.

- $1 \le n \le 10^5$, $1 \le q_i \le 10^9$, $n \le H \le 10^9$.

Orders q	H	**Answer k**
[3, 6, 7, 11]	8	4
[30, 11, 23, 4, 20]	5	30
[30, 11, 23, 4, 20]	6	23
[9, 12, 5]	10	3

Check the first row by hand. At $k = 4$: order 3 takes 1 hour, 6 takes 2, 7 takes 2, 11 takes 3. Total 8. At $k = 3$: $1 + 2 + 3 + 4 = 10 > 8$. So 4 is the smallest.

3.3 The checker: hours at a given speed

An order of x lots at speed k needs $\lceil x/k \rceil$ hours: "x divided by k, rounded **up**", because a partial hour still uses the hour. Total hours:

$$\text{hours}(k) = \sum_{i=0}^{n-1} \left\lceil \frac{q_i}{k} \right\rceil .$$

Read it out loud: "for each order, divide by the speed and round up, then add them all."

In integer arithmetic, rounding up is done with a trick: $\lceil x/k \rceil = \lfloor (x + k - 1)/k \rfloor$. With $x = 7$, $k = 4$: $(7 + 3)/4 = 10/4 = 2$ in integer division. Correct. With $x = 8$: $(8 + 3)/4 = 2$. Correct, no extra hour.

Here are the hours for the first example:

k	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**…**	**11**
hours	27	15	10	**8**	8	6	5	…	4
≤ 8?	no	no	no	**yes**	yes	yes	yes	…	yes

Hours never go **up** as k grows (each $\lceil q_i/k \rceil$ can only stay the same or shrink), so the "≤ 8 ?" row is a fence. The first yes is $k = 4$.

3.4 Brute force first

Try $k = 1, 2, 3, \ldots$ and stop at the first one that works. Each try costs $O(n)$. In the worst case the answer is near $\max q = 10^9$, so this is $O(n \cdot \max q) = 10^5 \times 10^9 = 10^{14}$ steps. Far too slow. The test file keeps it anyway, for checking on small inputs.

3.5 The insight: search the dial, not the list

What is the range of answers?

- $lo = 1$: you cannot go slower than 1 lot an hour.
- $hi = \max q$: at that speed every order takes exactly 1 hour, total $n \leq H$. Always fast enough. Going faster than $\max q$ never helps.

So the answer is the first true post on the fence over $[1, \max q]$. Binary search needs about $\log_2(10^9) \approx 30$ checks, each $O(n)$.

Time $O(n \log(\max q))$, about $10^5 \times 30 = 3 \times 10^6$ steps. **Space** $O(1)$ beyond the input.

Trace for [3, 6, 7, 11], $H = 8$ (the invariant: the answer is in $[lo, hi]$):

lo	hi	mid	hours(mid)	≤ 8?	action
1	11	6	6	yes	hi = 6
1	6	3	10	no	lo = 4
4	6	5	8	yes	hi = 5
4	5	4	8	yes	hi = 4
4	4				stop, answer 4

Four checks instead of up to eleven. For a dial up to 10^9, about 30 instead of a billion.

3.6 The trap: overflow

Two places where `int` (about $\pm 2.1 \times 10^9$) is not enough:

1. **The total hours.** At $k = 1$ with 10^5 orders of 10^9 lots, the total is 10^{14}. Use `long long` (about $\pm 9.2 \times 10^{18}$).
2. **The rounding-up trick.** $x + k - 1$ with $x = 10^9$ and $k = 10^9$ is about 2×10^9, just over the `int` limit. Doing it in `long long` keeps it safe.

The test file has a case with 10^5 orders of 10^9 lots, which overflows an `int` total. The answer there is $k = 100{,}000$: each order takes exactly 10,000 hours, total exactly 10^9.

3.7 C++ solution

```cpp
#include <algorithm>
#include <vector>
using namespace std;

// Hours needed at rate k: each order of size x takes ceil(x / k) hours.
long long hoursNeeded(const vector<int>& q, long long k) {
    long long total = 0;
    for (int x : q) {
        total += (x + k - 1) / k;          // ceil(x / k) in integer arithmetic
    }
    return total;
}

// Fence over k: "hoursNeeded(k) <= H" is false, ..., false, true, ..., true.
int minRate(const vector<int>& q, long long H) {
    long long lo = 1;
    long long hi = *max_element(q.begin(), q.end());   // always fast enough
    while (lo < hi) {                       // the answer lives in [lo, hi]
        long long mid = lo + (hi - lo) / 2;
        if (hoursNeeded(q, mid) <= H) {
            hi = mid;                       // mid is fast enough: keep it
        } else {
```

```
            lo = mid + 1;                   // too slow: answer is above mid
        }
    }
    return static_cast<int>(lo);
}
```

The full file `2026-11-06.cpp` adds the brute force, all the examples, a table of hours for $k = 1 \ldots 11$, a 100,000-case random cross-check against the brute force, and the overflow case. It compiles cleanly with `g++ -std=c++17 -O2 -Wall` and all tests pass.

3.8 Walkthrough of the C++

`long long hoursNeeded(const vector<int>& q, long long k).` The vector comes in by `const` reference (Lesson 2): no copy, and a promise not to change it. The return type is `long long` for the overflow reason above. Making `k` a `long long` too is the quiet hero of the function, as the next line shows.

`for (int x : q).` A range-based for loop: "for each element `x` of `q`". Each `x` is a copy, which is fine for an `int` (copying 4 bytes is as cheap as anything). For big objects you would write `const auto& x`.

`(x + k - 1) / k.` Here `x` is an `int` and `k` is a `long long`. When C++ mixes them in arithmetic, it first converts the smaller type up to the bigger one. This is called **integer promotion** (strictly, the "usual arithmetic conversions"). So the sum is done in `long long`, and $10^9 + 10^9 - 1$ does not overflow. If `k` were an `int`, the sum would be done in `int` and silently wrap around to a negative number. Signed overflow is **undefined behaviour** in C++, meaning the compiler is allowed to assume it never happens, and anything can go wrong. Division of two positive integers in C++ rounds **down** (toward zero), which is why the "+ k - 1" trick gives the round-up.

`*max_element(q.begin(), q.end()).` `max_element` (from `<algorithm>`) walks the range and returns an **iterator** (yesterday: a bookmark) to the biggest element. The `*` in front **dereferences** it: "give me the value at that bookmark". On an empty vector it would return `q.end()`, and dereferencing that is undefined behaviour; the constraints say $n \geq 1$, so we are safe, but in real code you would check first.

`long long hi = ....` The `int` returned by `*max_element` is converted to `long long` on assignment. Keeping `lo`, `hi` and `mid` as `long long` means `mid` goes into `hoursNeeded` with no conversion and no risk.

The loop. It is exactly yesterday's `firstTrue`, with the fence question written inline. `hi = mid` keeps a "yes" (it might be the first one), `lo = mid + 1` drops a "no". Because `lo <= mid < hi`, the range shrinks every time and the loop must end.

Could we reuse yesterday's template? Yes. With yesterday's `firstTrue` changed to take `long long` bounds, the whole function becomes one line:

```
return firstTrue(1LL, maxQ + 1, [&](long long k) { return hoursNeeded(q, k) <= H; });
```

The lambda captures `q` and `H` by reference (`[&]`) and asks the fence question. `1LL` is the number 1 written as a `long long` literal, so the template's types line up. This is the clean way to think about it: the search is always the same; only the question changes. (C++20 adds `std::ranges::partition_point` over `std::views::iota`, which does the same search over a range of numbers, but we stay in C++17.)

`static_cast<int>(lo)`. The answer is at most $\max q \le 10^9$, which fits in an `int`, so we convert back on purpose. `static_cast` makes the narrowing visible to a reader, and `-Wall` stays quiet.

The test file. `struct Case { vector<int> q; long long H; int expected; };` bundles one example. The list `cases = {{{3, 6, 7, 11}, 8, 4}, ...}` uses **brace initialisation**: the outer braces build the vector, each middle pair builds one `Case`, the inner braces build its `q`. `for (const auto& c : cases)` reads each case without copying it. The random test uses `mt19937` with a fixed seed (reproducible) and `uniform_int_distribution` to make small orders and every possible H from n to "more than enough", then `assert`s that the fast and brute force answers agree.

Common bugs in binary search on the answer (all avoided above):

- Setting hi too small (for example $hi = H$), so the true answer is outside the range.
- Using `int` for the total hours.
- Writing `x / k` (rounds down) instead of rounding up, which undercounts hours.
- Searching for the wrong end: we want the **smallest** good k, so we keep `hi = mid` on success.

Self-check. (1) For orders [9, 12, 5] and $H = 10$, what are hours(2) and hours(3)? (2) Why is $hi = \max q$ always safe? (3) A variant: "find the largest daily loss limit L such that …" where a bigger L makes the check *easier* to fail. How does the fence change?

Answers. (1) hours(2) $= 5 + 6 + 3 = 14$, hours(3) $= 3 + 4 + 2 = 9$; so the answer is 3. (2) At $k = \max q$ every order takes 1 hour, total $n \le H$. (3) The fence flips to "yes, yes, …, no, no". Search for the first "no" and step back one, or negate the question.

---

4. Music: Creedence Clearwater Revival, "Fortunate Son" (1969)

Context

"Fortunate Son" was written by **John Fogerty**, the singer, guitarist and songwriter of **Creedence Clearwater Revival** (CCR), a band from El Cerrito, California. It came out in the autumn of 1969 as a double A-side single with "Down on the Corner", and on the album **Willy and the Poor Boys**, released in November 1969. That year CCR released three albums, an astonishing pace that made them one of the biggest bands in America.

The story behind it

The song was written during the Vietnam War. Fogerty had himself been called up and served in the US Army Reserve in the late 1960s, so the draft was personal to him. He has said the spark was the wedding, in December 1968, of David Eisenhower, grandson of President Eisenhower, to Julie Nixon, daughter of the newly elected President Nixon. It made him think about the children of the powerful, who seemed never to be the ones sent to fight. He has also said he wrote the song very quickly, in about twenty minutes, sitting on the edge of his bed.

The song is not against soldiers. It is against **privilege**: the idea that some families send other people's sons to war while their own are protected by money and connections. Ironically, it has often been played at patriotic events by people who only hear the chorus's energy, and Fogerty has spoken more than once about how it gets misread.

Why people love it

It is short (under two and a half minutes), loud and furious, and there is not a wasted second. Fogerty's voice is raw and rasping, as if he is shouting across a field. The band plays with a simple, driving swamp-rock groove. It became one of the defining sounds of the Vietnam era and has been used in a huge number of films and TV shows set in that time. Rolling Stone and others regularly place it among the greatest songs of all time.

What to listen for

- **The drum opening.** Doug Clifford starts the song with a hard, simple pattern before the guitars crash in. It sets the tone in two seconds.
- **The guitar riff.** A bright, ringing, repeated figure that sounds almost cheerful, against words that are angry. That contrast is part of why the song works.
- **Fogerty's voice.** Listen to how he pushes it to the edge of breaking, especially at the end of each line. It sounds like real anger, not a performance of it.
- **How little happens.** No long solo, no bridge, no fade-out jam. Verse, chorus, verse, chorus, done. A lesson in saying one thing and stopping.

Next: Sunday (2026-11-08) is a revision day for all three tracks. Saturday continues with Lesson 11 in probability (random variables and the PMF), Lesson 12 in derivatives (interest rate swaps), and Lesson 12 in algorithms (stacks).